МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»

Лабораторная работа № 4 по курсу «Операционные системы и архитектура компьютера»

Выполнил: Новак Дмитрий Евгеньевич
Группа: М8О-201БВ-25
Преподаватель: Миронов Евгений Сергеевич
Дата: 12.11.2026
Оценка: _____

Москва, 2026

Условие

Цель работы: освоить принципы работы с файловыми системами и получить практические навыки обмена данными между процессами посредством отображаемых в память файлов (memory-mapped files).

Задание: составить и отладить программу на языке C, работающую с процессами в ОС семейства UNIX. Основной процесс должен создать дочерний процесс для решения задачи. Межпроцессное взаимодействие организуется через отображаемые файлы и системные сигналы. Необходимо обрабатывать системные ошибки, которые могут возникнуть при выполнении программы. Вариант задачи сохраняется из лабораторной работы №2.

Вариант 9: пользователь вводит имя файла в родительский процесс. Родитель открывает файл на чтение и перенаправляет его в стандартный ввод ребёнка. Каждая строка файла содержит произвольное количество чисел типа `float`. Дочерний процесс последовательно делит первое число на все следующие числа строки. Результат передаётся родителю, который выводит его в свой `stdout`. При делении на ноль проверка выполняется в дочернем процессе, после чего оба процесса завершают работу.

Метод решения

Родительская и дочерняя части реализованы отдельными программами: `lab4_parent` и `lab4_child`. В отличие от лабораторной работы №2, результаты передаются через общую область памяти, связанную с временным файлом. Сигналы используются для уведомления о готовности результата и подтверждения его получения.

Родитель открывает входной файл и создаёт временный файл функцией `mkstemp()`. Вызов `ftruncate()` задаёт размер временного файла, равный `sizeof(float)`. Затем `mmap()` с флагом `MAP_SHARED` создаёт отображение файла в память. В используемой системе размер `float` равен четырём байтам, поэтому в общей области хранится один результат.

Имя временного файла удаляется через `unlink()` сразу после его создания. Это не уничтожает используемый файл: он остаётся доступен через открытый дескриптор и отображения. После освобождения всех ссылок ОС удаляет его содержимое, поэтому отдельный файл с результатами в каталоге не остаётся.

После `fork()` дочерняя ветвь перенаправляет входной файл в `stdin` через `dup2()` и выполняет `execl()`. Дескриптор временного файла наследуется ребёнком и передаётся новой программе как аргумент командной строки. После `exec` ребёнок создаёт собственное отображение этого же файла. Виртуальные адреса отображений у процессов могут различаться.

Дочерний процесс читает строки через `getline()`, преобразует числа функцией `strtof()` и выполняет деление. Пустые строки пропускаются; строка с одним числом возвращает это число. Нулевой делитель проверяется перед операцией деления, а некорректные числа и выход результата за диапазон `float` приводят к сообщению об ошибке и завершению.

Обмен результатами выполняется в следующем порядке:

1. Ребёнок записывает вычисленный результат в общую область: `*shared = result`.

2. Ребёнок отправляет родителю SIGUSR1 и ожидает подтверждение через sigwait().
3. Родитель получает уведомление, читает *shared, выводит значение и отправляет ребёнку SIGUSR2.
4. После подтверждения ребёнок переходит к следующей строке и может перезаписать общую ячейку.

Такое подтверждение не позволяет ребёнку перезаписать результат до того, как его прочитает родитель. Сами числа не передаются внутри сигналов: сигналы несут только уведомления.

Родитель блокирует SIGUSR1 и SIGCHLD до создания ребёнка. Обработчик сигналов устанавливает флаги result_ready и child_finished. Основной цикл родителя проверяет эти флаги, а sigsuspend() временно разрешает доставку сигналов и приостанавливает выполнение. Ребёнок блокирует SIGUSR2 до первой записи и принимает подтверждения через sigwait(). Ранний сигнал остаётся ожидающим, поэтому уведомление не теряется между проверкой состояния и ожиданием.

При окончании файла ребёнок освобождает отображение и завершается. Родитель получает SIGCHLD, вызывает waitpid() и возвращает код завершения ребёнка. Затем освобождается отображение родителя. При ошибке родителя после запуска ребёнка выполняется принудительное завершение ребёнка и ожидание его статуса.

Описание программы

Структура программной части

Таблица 1 — Файлы программной части

Файл	Назначение
src/parent.c	Ввод имени файла, создание отображаемого файла и ребёнка, приём уведомлений, вывод результатов и ожидание завершения.
src/child.c	Чтение строк из стандартного ввода, деление чисел, запись результата в общую память и ожидание подтверждения.
CMakeLists.txt	Сборка исполняемых файлов lab4_parent и lab4_child в стандарте C11.
data/*.txt	Примеры корректного ввода, отрицательных чисел и деления на ноль.
report/trace/*.txt	Полная и сокращённая трассировки выполнения на Ubuntu ARM64.

Основные переменные и функции

Вся основная логика каждой программы находится в её main(). В родительской программе отдельно определён обработчик handle_signal(), который устанавливает флаг готовности результата при SIGUSR1 и флаг завершения ребёнка при SIGCHLD. Вывод и работа с общей памятью выполняются основным кодом, а не внутри обработчика.

Переменная `input_fd` хранит дескриптор входного файла, `shared_fd` — дескриптор отображаемого файла, `shared` — адрес отображения. После создания отображения дескриптор можно закрыть: отображение остаётся действительным до `munmap()`.

Дочерний процесс хранит текущую строку в динамическом буфере `line`, который переиспользуется функцией `getline()` и освобождается через `free()`. Переменная `result` имеет тип `float`; `line_number` используется в сообщениях об ошибках.

Коды завершения: 0 — успех, 1 — общая или системная ошибка, 2 — деление на ноль, 3 — некорректные входные числа или выход результата за диапазон `float`, 4 — ошибка запуска дочерней программы через `exec`.

Системные вызовы и POSIX-интерфейсы

Таблица 2 — Используемые системные вызовы и POSIX-интерфейсы

Вызов	Назначение в программе
`open()`	Открытие входного файла только для чтения.
`mkstemp()`, `unlink()`	Создание временного файла и удаление его имени из каталога.
`ftruncate()`	Установка размера отображаемого файла.
`mmap()`, `munmap()`	Создание и освобождение отображения файла в память.
`fork()`, `execl()`	Создание ребёнка и запуск дочерней программы.
`dup2()`, `close()`	Перенаправление входного файла в `stdin` и закрытие дескрипторов.
`sigaction()`	Установка обработчиков `SIGUSR1` и `SIGCHLD`.
`sigprocmask()`	Блокировка сигналов до их обработки или ожидания.
`sigsuspend()`, `sigwait()`	Ожидание уведомления у родителя и подтверждения у ребёнка.
`kill()`	Отправка сигналов другому процессу.
`waitpid()`	Получение статуса завершения дочернего процесса.

`getline()`, `strtof()`, `printf()` и `fflush()` относятся к стандартной библиотеке C. Имена функций в исходном коде могут отличаться от системных вызовов, которые видны в `strace`.

Ключевые фрагменты кода

Родитель передаёт дескриптор отображаемого файла строковым аргументом и отображает область размером в один `float`:

Листинг 1 — Подготовка дескриптора и создание отображения (parent.c)

```
    char descriptor[32];
    snprintf(descriptor, sizeof(descriptor), "%d", shared_fd);

    float* shared = mmap(NULL, sizeof(float), PROT_READ |
     ↪  PROT_WRITE, MAP_SHARED, shared_fd, 0);
```

Ребёнок записывает число в общую память, отправляет уведомление и ждёт подтверждения. Операция записи в `*shared` является обычным обращением к памяти и не требует отдельного вызова `write()`.

Листинг 2 — Передача результата и ожидание подтверждения (child.c)

```
        *shared = result;

        if (kill(parent_pid, SIGUSR1) == -1) {
            perror("kill");
            status = GENERAL_ERROR_EXIT_CODE;
            break;
        }

        int received;
        int error = sigwait(&signals, &received);

        if (error != 0) {
            fprintf(stderr, "Ошибка sigwait: %d\n", error);
            status = GENERAL_ERROR_EXIT_CODE;
            break;
        }
```

Родитель ожидает сигнал, выводит доступное значение и подтверждает его получение:

Листинг 3 — Ожидание и чтение результата (parent.c)

```
    while (!parent_error) {
        while (!result_ready && !child_finished) {
            if (sigsuspend(&wait_mask) == -1 && errno != EINTR) {
                perror("sigsuspend");
                parent_error = 1;
                break;
            }
        }

        if (parent_error) {
            break;
        }

        if (result_ready) {
            result_ready = 0;

            printf("%g\n", *shared);

            if (fflush(stdout) == EOF) {
                perror("fflush");
                parent_error = 1;
                break;
            }

            if (kill(pid, SIGUSR2) == -1) {
                perror("kill");
                parent_error = 1;
                break;
            }
        }
```

Инструкция по запуску

Сборка из корня репозитория с помощью CMake:

```
$ cmake -S lab_04 -B build/lab_04
$ cmake --build build/lab_04
```

Для запуска нужно перейти в каталог сборки: дочерняя программа ищется по относительному пути `./lab4_child`.

```
$ cd build/lab_04
$ ./lab4_parent
Введите путь к файлу: ../../lab_04/data/ok.txt
0.5
20
10
100000
500
0.05
0
```

Первая строка файла содержит `32 2 2 2 2 2 2`. Результат вычисления:

$$32/2/2/2/2/2/2 = 0{,}5.$$

Строка `10` возвращает число без изменения. При вычислениях используется тип `float`, а для вывода — формат `%g`.

Для файла с отрицательными числами получен следующий результат:

```
$ ./lab4_parent
Введите путь к файлу: ../../lab_04/data/negatives.txt
0.1
-100
-1000
```

При нулевом делителе дочерний процесс сообщает об ошибке, а оба процесса завершаются с кодом 2:

```
$ ./lab4_parent
Введите путь к файлу: ../../lab_04/data/zero.txt
Ошибка в строке 1: деление на ноль.
$ echo $?
2
```

Родитель читает из своего `stdin` именно имя файла. Чтобы выполнить запуск без ручного ввода пути, можно передать строку через оболочку:

```
$ printf '../../lab_04/data/ok.txt\n' | ./lab4_parent
```

В данном случае канал создаёт оболочка только для передачи имени файла родителю. Обмен результатами между родителем и ребёнком по-прежнему выполняется через `mmap`.

Демонстрация работы strace

Полная трассировка получена в Ubuntu ARM64. Команды выполняются из корня репозитория. Ключ `-f` включает наблюдение за дочерним процессом, а `-o` сохраняет системные вызовы и сигналы в файл.

```
$ mkdir -p lab_04/report/trace
$ cd build/lab_04
$ printf '../../lab_04/data/ok.txt\n' | strace -f -o
↪  ../../lab_04/report/trace/strace.txt ./lab4_parent
```

В данной трассировке PID родителя равен 5028, PID ребёнка — 5029. Ниже приведены выбранные строки из того же запуска; полная трассировка находится в приложении Б.

Листинг 4 — Ключевые системные вызовы из strace

```
5028  rt_sigaction(SIGUSR1, {sa_handler=0xb4f908eb1128, sa_mask=[],
↪  sa_flags=SA_NOCLDSTOP}, NULL, 8) = 0
5028  rt_sigaction(SIGCHLD, {sa_handler=0xb4f908eb1128, sa_mask=[],
↪  sa_flags=SA_NOCLDSTOP}, NULL, 8) = 0
5028  rt_sigprocmask(SIG_BLOCK, [USR1 CHLD], [], 8) = 0

5028  openat(AT_FDCWD, "../../lab_04/data/ok.txt", O_RDONLY) = 3

5028  openat(AT_FDCWD, "/tmp/lab4-SjREMh", O_RDWR|O_CREAT|O_EXCL, 0600)
↪  = 4
5028  unlinkat(AT_FDCWD, "/tmp/lab4-SjREMh", 0) = 0
5028  ftruncate(4, 4)                   = 0
5028  mmap(NULL, 4, PROT_READ|PROT_WRITE, MAP_SHARED, 4, 0) =
↪  0xfde6c82a9000

5028  clone(child_stack=NULL,
↪  flags=CLONE_CHILD_CLEARTID|CLONE_CHILD_SETTID|SIGCHLD,
↪  child_tidptr=0xfde6c82af030) = 5029

5029  dup3(3, 0, 0)                     = 0
5029  close(3)                          = 0
5029  execve("./lab4_child", ["lab4_child", "4"], 0xfffff2df75a8 /* 65
↪  vars */) = 0

5029  getppid()                         = 5028
5029  rt_sigprocmask(SIG_BLOCK, [USR2], NULL, 8) = 0
5029  mmap(NULL, 4, PROT_READ|PROT_WRITE, MAP_SHARED, 4, 0) =
↪  0xfd7005278000
5029  close(4)                          = 0

5029  read(0, "32 2 2 2 2 2 2\n100 5\n10\n1000 0.0"..., 4096) = 64
5029  kill(5028, SIGUSR1 <unfinished ...>
5028  <... rt_sigsuspend resumed>)      = ? ERESTARTNOHAND (To be
↪  restarted if no handler)
5029  <... kill resumed>)               = 0
5028  --- SIGUSR1 {si_signo=SIGUSR1, si_code=SI_USER, si_pid=5029,
↪  si_uid=1000} ---
5028  rt_sigreturn({mask=[USR1 CHLD]} <unfinished ...>
5029  rt_sigtimedwait([USR2] <unfinished ...>
5028  <... rt_sigreturn resumed>)       = -1 EINTR (Interrupted system
↪  call)
5028  write(1, "0.5\n", 4)              = 4
5028  kill(5029, SIGUSR2)               = 0
```

```
5029  <... rt_sigtimedwait resumed>, {si_signo=SIGUSR2, si_code=SI_USER,
↪  si_pid=5028, si_uid=1000}, NULL, 8) = 12 (SIGUSR2)

5029  munmap(0xfd7005278000, 4)         = 0
5028  rt_sigsuspend([], 8 <unfinished ...>
5029  exit_group(0)                     = ?
5029  +++ exited with 0 +++
5028  <... rt_sigsuspend resumed>)      = ? ERESTARTNOHAND (To be
↪  restarted if no handler)
5028  --- SIGCHLD {si_signo=SIGCHLD, si_code=CLD_EXITED, si_pid=5029,
↪  si_uid=1000, si_status=0, si_utime=0, si_stime=0} ---
5028  rt_sigreturn({mask=[USR1 CHLD]})  = -1 EINTR (Interrupted system
↪  call)
5028  wait4(5029, [{WIFEXITED(s) && WEXITSTATUS(s) == 0}], 0, NULL) =
↪  5029
5028  munmap(0xfde6c82a9000, 4)         = 0
5028  exit_group(0)                     = ?
5028  +++ exited with 0 +++
```

Входному файлу соответствует дескриптор 3, временному — 4. Размер временного файла составляет четыре байта. Флаг `MAP_SHARED` обеспечивает общий доступ к содержимому одного файла, хотя адреса отображений различаются. Закрытие дескриптора после `mmap()` не отменяет отображение.

В этой системе функциям `fork()`, `dup2()`, `execl()` и `waitpid()` соответствуют системные вызовы `clone()`, `dup3()`, `execve()` и `wait4()`. Запуск `lab4_child` через `exec` не меняет PID 5029.

Ребёнок записывает первый результат в общую память и отправляет родителю `SIGUSR1`. Родитель выходит из `rt_sigsuspend()`, выводит `0.5` через `write(1, ...)` и подтверждает получение сигналом `SIGUSR2`. Ожиданию `sigwait()` у ребёнка соответствует `rt_sigtimedwait()`.

Вычисление и запись в `*shared` не являются системными вызовами и отдельно в трассировке не видны. Большая часть других вызовов `mmap` относится к загрузке библиотек и служебной памяти.

Пометки `unfinished` и `resumed` разделяют начало и продолжение одного вызова. Возврат `EINTR` при доставке сигнала здесь означает нормальное прерывание ожидания.

В конце ребёнок освобождает отображение и завершается. Родитель получает `SIGCHLD`, забирает статус через `wait4()` и освобождает своё отображение. Оба процесса завершаются успешно: в трассировке присутствуют две строки `exited with 0`.

Выводы

Таким образом, данная лабораторная работа никак кардинально не изменила мою жизнь: я сделал свою версию 3-way handshake(у меня бюджетная 2-way) как в TCP, поработал с mmap, который занимает половину вывода strace, и научил два процесса по очереди пользоваться четырьмя байтами памяти. Также выяснил, что kill не обязательно кого-то убивает, иногда он просто сообщает, что число готово.

Приложение А. Исходный код программы

Листинг А.1 — Файл parent.c

```
#include <errno.h>
#include <fcntl.h>
#include <signal.h>
#include <stdio.h>
#include <stdlib.h>
#include <sys/mman.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <unistd.h>

#define GENERAL_ERROR_EXIT_CODE 1
#define DIVISION_BY_ZERO_EXIT_CODE 2
#define INPUT_ERROR_EXIT_CODE 3
#define EXEC_ERROR_EXIT_CODE 4

sig_atomic_t result_ready = 0;
sig_atomic_t child_finished = 0;

void handle_signal(int signal_number) {
    if (signal_number == SIGUSR1) {
        result_ready = 1;
    } else if (signal_number == SIGCHLD) {
        child_finished = 1;
    }
}

int main(void) {
    struct sigaction action = {0};
    action.sa_handler = handle_signal;
    action.sa_flags = SA_NOCLDSTOP;

    if (sigaction(SIGUSR1, &action, NULL) == -1 || sigaction(SIGCHLD,
    ↪  &action, NULL) == -1) {
        perror("sigaction");
        return GENERAL_ERROR_EXIT_CODE;
    }

    sigset_t signals;
    sigset_t wait_mask;

    if (sigemptyset(&signals) == -1 || sigaddset(&signals, SIGUSR1) ==
    ↪  -1 || sigaddset(&signals, SIGCHLD) == -1) {
        perror("sigset");
        return GENERAL_ERROR_EXIT_CODE;
    }

    if (sigprocmask(SIG_BLOCK, &signals, &wait_mask) == -1) {
        perror("sigprocmask");
        return GENERAL_ERROR_EXIT_CODE;
    }

    if (sigdelset(&wait_mask, SIGUSR1) == -1 || sigdelset(&wait_mask,
    ↪  SIGCHLD) == -1) {
        perror("sigdelset");
        return GENERAL_ERROR_EXIT_CODE;
    }

    char* filename = NULL;
    size_t capacity = 0;

    printf("Введите путь к файлу: ");
```

```
    if (fflush(stdout) == EOF) {
        perror("fflush");
        return GENERAL_ERROR_EXIT_CODE;
    }

    ssize_t length = getline(&filename, &capacity, stdin);

    if (length == -1) {
        if (ferror(stdin)) {
            perror("getline");
        } else {
            fprintf(stderr, "\nОшибка: путь к файлу не введен.\n");
        }

        free(filename);
        return GENERAL_ERROR_EXIT_CODE;
    }

    while (length > 0 && filename[length - 1] == '\n') {
        filename[--length] = '\0';
    }

    if (length == 0) {
        fprintf(stderr, "Ошибка: путь к файлу не может быть пустым.\n");
        free(filename);
        return GENERAL_ERROR_EXIT_CODE;
    }

    int input_fd = open(filename, O_RDONLY);
    free(filename);

    if (input_fd == -1) {
        perror("open");
        return GENERAL_ERROR_EXIT_CODE;
    }

    char temporary_path[] = "/tmp/lab4-XXXXXX";
    int shared_fd = mkstemp(temporary_path);

    if (shared_fd == -1) {
        perror("mkstemp");

        if (close(input_fd) == -1) {
            perror("close");
        }

        return GENERAL_ERROR_EXIT_CODE;
    }

    if (unlink(temporary_path) == -1) {
        perror("unlink");

        if (close(input_fd) == -1) {
            perror("close");
        }

        if (close(shared_fd) == -1) {
            perror("close");
        }

        return GENERAL_ERROR_EXIT_CODE;
    }
```

```
    if (ftruncate(shared_fd, sizeof(float)) == -1) {
        perror("ftruncate");

        if (close(input_fd) == -1) {
            perror("close");
        }

        if (close(shared_fd) == -1) {
            perror("close");
        }

        return GENERAL_ERROR_EXIT_CODE;
    }

    char descriptor[32];
    snprintf(descriptor, sizeof(descriptor), "%d", shared_fd);

    float* shared = mmap(NULL, sizeof(float), PROT_READ | PROT_WRITE,
    ↪  MAP_SHARED, shared_fd, 0);

    if (shared == MAP_FAILED) {
        perror("mmap");

        if (close(input_fd) == -1) {
            perror("close");
        }

        if (close(shared_fd) == -1) {
            perror("close");
        }

        return GENERAL_ERROR_EXIT_CODE;
    }

    pid_t pid = fork();

    if (pid == -1) {
        perror("fork");

        if (close(input_fd) == -1) {
            perror("close");
        }

        if (close(shared_fd) == -1) {
            perror("close");
        }

        if (munmap(shared, sizeof(float)) == -1) {
            perror("munmap");
        }

        return GENERAL_ERROR_EXIT_CODE;
    }

    if (pid == 0) {
        if (dup2(input_fd, STDIN_FILENO) == -1) {
            perror("dup2");
            exit(GENERAL_ERROR_EXIT_CODE);
        }

        if (input_fd != STDIN_FILENO && close(input_fd) == -1) {
            perror("close");
            exit(GENERAL_ERROR_EXIT_CODE);
        }
```

```
    execl("./lab4_child", "lab4_child", descriptor, (char*)NULL);

    perror("execl");
    exit(EXEC_ERROR_EXIT_CODE);
}

int parent_error = 0;

if (close(input_fd) == -1) {
    perror("close");
    parent_error = 1;
}

if (close(shared_fd) == -1) {
    perror("close");
    parent_error = 1;
}

while (!parent_error) {
    while (!result_ready && !child_finished) {
        if (sigsuspend(&wait_mask) == -1 && errno != EINTR) {
            perror("sigsuspend");
            parent_error = 1;
            break;
        }
    }

    if (parent_error) {
        break;
    }

    if (result_ready) {
        result_ready = 0;

        printf("%g\n", *shared);

        if (fflush(stdout) == EOF) {
            perror("fflush");
            parent_error = 1;
            break;
        }

        if (kill(pid, SIGUSR2) == -1) {
            perror("kill");
            parent_error = 1;
            break;
        }
    }

    if (child_finished) {
        break;
    }
}

if (parent_error && kill(pid, SIGKILL) == -1 && errno != ESRCH) {
    perror("kill");
}

int status = 0;

while (waitpid(pid, &status, 0) == -1) {
    if (errno == EINTR) {
        continue;
```

```
        }

        perror("waitpid");
        parent_error = 1;
        break;
    }

    int result = GENERAL_ERROR_EXIT_CODE;

    if (!parent_error) {
        if (WIFEXITED(status)) {
            result = WEXITSTATUS(status);

            if (result == EXEC_ERROR_EXIT_CODE) {
                fprintf(stderr, "Ошибка: не удалось запустить дочерний
                ↪  процесс.\n");
            } else if (result != 0 && result !=
            ↪  DIVISION_BY_ZERO_EXIT_CODE && result !=
            ↪  INPUT_ERROR_EXIT_CODE) {
                fprintf(stderr, "Дочерний процесс завершился с кодом
                ↪  %d.\n", result);
            }
        } else if (WIFSIGNALED(status)) {
            fprintf(stderr, "Дочерний процесс завершён сигналом %d.\n",
            ↪  WTERMSIG(status));
        } else {
            fprintf(stderr, "Ошибка: дочерний процесс завершился в
            ↪  неизвестном состоянии.\n");
        }
    }

    if (munmap(shared, sizeof(float)) == -1) {
        perror("munmap");
        result = GENERAL_ERROR_EXIT_CODE;
    }

    return result;
}
```

Листинг А.2 — Файл child.c

```
#include <ctype.h>
#include <errno.h>
#include <math.h>
#include <signal.h>
#include <stdio.h>
#include <stdlib.h>
#include <sys/mman.h>
#include <unistd.h>

#define GENERAL_ERROR_EXIT_CODE 1
#define DIVISION_BY_ZERO_EXIT_CODE 2
#define INPUT_ERROR_EXIT_CODE 3

int main(int argc, char* argv[]) {
    if (argc != 2) {
        fprintf(stderr, "Ошибка: неверное количество аргументов.\n");
        return GENERAL_ERROR_EXIT_CODE;
    }

    int shared_fd = (int)strtol(argv[1], NULL, 10);
    pid_t parent_pid = getppid();
    sigset_t signals;
```

```
    if (sigemptyset(&signals) == -1 || sigaddset(&signals, SIGUSR2) ==
    ↪  -1) {
        perror("sigset");
        return GENERAL_ERROR_EXIT_CODE;
    }

    if (sigprocmask(SIG_BLOCK, &signals, NULL) == -1) {
        perror("sigprocmask");
        return GENERAL_ERROR_EXIT_CODE;
    }

    float* shared = mmap(NULL, sizeof(float), PROT_READ | PROT_WRITE,
    ↪  MAP_SHARED, shared_fd, 0);

    if (shared == MAP_FAILED) {
        perror("mmap");
        return GENERAL_ERROR_EXIT_CODE;
    }

    if (close(shared_fd) == -1) {
        perror("close");

        if (munmap(shared, sizeof(float)) == -1) {
            perror("munmap");
        }

        return GENERAL_ERROR_EXIT_CODE;
    }

    char* line = NULL;
    size_t capacity = 0;
    size_t line_number = 0;
    int status = 0;

    while (getline(&line, &capacity, stdin) != -1) {
        line_number++;

        char* current = line;

        while (isspace((unsigned char)*current)) {
            current++;
        }

        if (*current == '\0') {
            continue;
        }

        char* end = NULL;
        errno = 0;
        float result = strtof(current, &end);

        if (current == end || errno == ERANGE || !isfinite(result) ||
        ↪  (*end != '\0' && !isspace((unsigned char)*end))) {
            fprintf(stderr, "Ошибка в строке %zu: некорректное
            ↪  число.\n", line_number);
            status = INPUT_ERROR_EXIT_CODE;
            break;
        }

        current = end;

        while (1) {
            while (isspace((unsigned char)*current)) {
                current++;
```

```
            }

            if (*current == '\0') {
                break;
            }

            errno = 0;
            float divisor = strtof(current, &end);

            if (current == end || errno == ERANGE || !isfinite(divisor)
            ↪  || (*end != '\0' && !isspace((unsigned char)*end))) {
                fprintf(stderr, "Ошибка в строке %zu: некорректное
                ↪  число.\n", line_number);
                status = INPUT_ERROR_EXIT_CODE;
                break;
            }

            if (divisor == 0.0f) {
                fprintf(stderr, "Ошибка в строке %zu: деление на
                ↪  ноль.\n", line_number);
                status = DIVISION_BY_ZERO_EXIT_CODE;
                break;
            }

            result /= divisor;

            if (!isfinite(result)) {
                fprintf(stderr, "Ошибка в строке %zu: результат
                ↪  вычисления вышел за диапазон float.\n", line_number);
                status = INPUT_ERROR_EXIT_CODE;
                break;
            }

            current = end;
        }

        if (status != 0) {
            break;
        }

        *shared = result;

        if (kill(parent_pid, SIGUSR1) == -1) {
            perror("kill");
            status = GENERAL_ERROR_EXIT_CODE;
            break;
        }

        int received;
        int error = sigwait(&signals, &received);

        if (error != 0) {
            fprintf(stderr, "Ошибка sigwait: %d\n", error);
            status = GENERAL_ERROR_EXIT_CODE;
            break;
        }
    }

    if (ferror(stdin)) {
        perror("getline");
        status = GENERAL_ERROR_EXIT_CODE;
    }

    free(line);
```

```
    if (munmap(shared, sizeof(float)) == -1) {
        perror("munmap");
        status = GENERAL_ERROR_EXIT_CODE;
    }

    return status;
}
```

Листинг А.3 — Файл CMakeLists.txt

```
cmake_minimum_required(VERSION 3.20)

project(lab_04_variant_9 C)

set(CMAKE_C_STANDARD 11)
set(CMAKE_C_STANDARD_REQUIRED ON)
set(CMAKE_C_EXTENSIONS ON)

add_compile_options(-Wall -Wextra -Wpedantic)

add_executable(lab4_parent src/parent.c)
add_executable(lab4_child src/child.c)
```

Приложение Б. Полный вывод strace

Полный вывод получен при запуске `lab4_parent` в Ubuntu ARM64 с параметром `-f`. Трассировка включает родительский и дочерний процессы, загрузку библиотек, обмен сигналами и завершение.

Листинг Б.1 — Полный вывод strace -f ./lab4_parent

```
5028  execve("./lab4_parent", ["./lab4_parent"], 0xffffdd4774a8 /* 65
↪  vars */) = 0
5028  brk(NULL)                           = 0xb4f930cef000
5028  mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS,
↪  -1, 0) = 0xfde6c82ae000
5028  faccessat(AT_FDCWD, "/etc/ld.so.preload", R_OK) = -1 ENOENT (No
↪  such file or directory)
5028  openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
5028  fstat(3, {st_mode=S_IFREG|0644, st_size=63707, ...}) = 0
5028  mmap(NULL, 63707, PROT_READ, MAP_PRIVATE, 3, 0) = 0xfde6c8268000
5028  close(3)                            = 0
5028  openat(AT_FDCWD, "/usr/lib/aarch64-linux-gnu/libc.so.6",
↪  O_RDONLY|O_CLOEXEC) = 3
5028  read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\3\0\267\0\1\0\0\0\3101\2\⌋
↪  0\0\0\0\0"..., 832) =
↪  832
5028  fstat(3, {st_mode=S_IFREG|0755, st_size=1788240, ...}) = 0
5028  mmap(NULL, 1957440, PROT_NONE,
↪  MAP_PRIVATE|MAP_ANONYMOUS|MAP_DENYWRITE, -1, 0) = 0xfde6c808a000
5028  mmap(0xfde6c8090000, 1891904, PROT_READ|PROT_EXEC,
↪  MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0) = 0xfde6c8090000
5028  munmap(0xfde6c808a000, 24576)       = 0
5028  munmap(0xfde6c825e000, 40512)       = 0
5028  mprotect(0xfde6c8239000, 81920, PROT_NONE) = 0
5028  mmap(0xfde6c824d000, 20480, PROT_READ|PROT_WRITE,
↪  MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x1ad000) = 0xfde6c824d000
5028  mmap(0xfde6c8252000, 48704, PROT_READ|PROT_WRITE,
↪  MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1, 0) = 0xfde6c8252000
5028  mmap(0xfde6c8090000, 1740800, PROT_READ|PROT_EXEC|0x10,
↪  MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0) = 0xfde6c8090000
5028  close(3)                            = 0
5028  set_tid_address(0xfde6c82af388)     = 5028
5028  set_robust_list(0xfde6c82af040, 24) = 0
5028  rseq(0xfde6c82af740, 0x21, 0, 0xd428bc00) = 0
5028  mprotect(0xfde6c824d000, 12288, PROT_READ) = 0
5028  mprotect(0xb4f908ecf000, 4096, PROT_READ) = 0
5028  mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS,
↪  -1, 0) = 0xfde6c82ac000
5028  mprotect(0xfde6c82b6000, 8192, PROT_READ) = 0
5028  prlimit64(0, RLIMIT_STACK, NULL, {rlim_cur=8192*1024,
↪  rlim_max=RLIM64_INFINITY}) = 0
5028  getrandom("\xe8\xd7\x90\x09\xb3\xc4\x03\x06", 8, GRND_NONBLOCK) = 8
5028  munmap(0xfde6c8268000, 63707)       = 0
5028  rt_sigaction(SIGUSR1, {sa_handler=0xb4f908eb1128, sa_mask=[],
↪  sa_flags=SA_NOCLDSTOP}, NULL, 8) = 0
5028  rt_sigaction(SIGCHLD, {sa_handler=0xb4f908eb1128, sa_mask=[],
↪  sa_flags=SA_NOCLDSTOP}, NULL, 8) = 0
5028  rt_sigprocmask(SIG_BLOCK, [USR1 CHLD], [], 8) = 0
5028  fstat(1, {st_mode=S_IFCHR|0600, st_rdev=makedev(0x88, 0), ...}) = 0
5028  brk(NULL)                           = 0xb4f930cef000
5028  brk(0xb4f930e00000)                 = 0xb4f930e00000
```

```
5028  write(1, "\320\222\320\262\320\265\320\264\320\270\321\202\320\265
↪  \320\277\321\203\321\202\321\214 \320\272 \321\204\320\260\320"...,
↪  39) = 39
5028  fstat(0, {st_mode=S_IFIFO|0600, st_size=0, ...}) = 0
5028  read(0, "../../lab_04/data/ok.txt\n", 4096) = 25
5028  openat(AT_FDCWD, "../../lab_04/data/ok.txt", O_RDONLY) = 3
5028  rt_sigprocmask(SIG_BLOCK, ~[], [USR1 CHLD], 8) = 0
5028  mmap(NULL, 4096, PROT_READ|PROT_WRITE,
↪  MAP_DROPPABLE|MAP_ANONYMOUS, -1, 0) = 0xfde6c82ab000
5028  mmap(NULL, 4096, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS,
↪  -1, 0) = 0xfde6c82aa000
5028  rt_sigprocmask(SIG_SETMASK, [USR1 CHLD], NULL, 8) = 0
5028  getrandom("\xfb\x6d\xae\x9b\x2c\x60\x32\xc6\x15\x63\xbf\xd7\xce\xb⌋
↪  0\x33\x50\x20\x57\xe7\xcd\xa7\x29\xe8\xa8\xca\x2b\x34\xf1\xc8\x54\xb⌋
↪  2\x83", 32, 0) =
↪  32
5028  openat(AT_FDCWD, "/tmp/lab4-SjREMh", O_RDWR|O_CREAT|O_EXCL, 0600)
↪  = 4
5028  unlinkat(AT_FDCWD, "/tmp/lab4-SjREMh", 0) = 0
5028  ftruncate(4, 4)                   = 0
5028  mmap(NULL, 4, PROT_READ|PROT_WRITE, MAP_SHARED, 4, 0) =
↪  0xfde6c82a9000
5028  rt_sigprocmask(SIG_BLOCK, ~[], [USR1 CHLD], 8) = 0
5028  clone(child_stack=NULL,
↪  flags=CLONE_CHILD_CLEARTID|CLONE_CHILD_SETTID|SIGCHLD,
↪  child_tidptr=0xfde6c82af030) = 5029
5029  set_robust_list(0xfde6c82af040, 24 <unfinished ...>
5028  rt_sigprocmask(SIG_SETMASK, [USR1 CHLD] <unfinished ...>
5029  <... set_robust_list resumed>)    = 0
5028  <... rt_sigprocmask resumed>, NULL, 8) = 0
5029  rt_sigprocmask(SIG_SETMASK, [USR1 CHLD] <unfinished ...>
5028  close(3)                          = 0
5028  close(4 <unfinished ...>
5029  <... rt_sigprocmask resumed>, NULL, 8) = 0
5028  <... close resumed>)              = 0
5028  rt_sigsuspend([], 8 <unfinished ...>
5029  dup3(3, 0, 0)                     = 0
5029  close(3)                          = 0
5029  execve("./lab4_child", ["lab4_child", "4"], 0xfffff2df75a8 /* 65
↪  vars */) = 0
5029  brk(NULL)                         = 0xc01ede8bb000
5029  mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS,
↪  -1, 0) = 0xfd700527b000
5029  faccessat(AT_FDCWD, "/etc/ld.so.preload", R_OK) = -1 ENOENT (No
↪  such file or directory)
5029  openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
5029  fstat(3, {st_mode=S_IFREG|0644, st_size=63707, ...}) = 0
5029  mmap(NULL, 63707, PROT_READ, MAP_PRIVATE, 3, 0) = 0xfd7005235000
5029  close(3)                          = 0
5029  openat(AT_FDCWD, "/usr/lib/aarch64-linux-gnu/libc.so.6",
↪  O_RDONLY|O_CLOEXEC) = 3
5029  read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\3\0\267\0\1\0\0\0\3101\2\⌋
↪  0\0\0\0\0"..., 832) =
↪  832
5029  fstat(3, {st_mode=S_IFREG|0755, st_size=1788240, ...}) = 0
5029  mmap(NULL, 1957440, PROT_NONE,
↪  MAP_PRIVATE|MAP_ANONYMOUS|MAP_DENYWRITE, -1, 0) = 0xfd7005057000
5029  mmap(0xfd7005060000, 1891904, PROT_READ|PROT_EXEC,
↪  MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0) = 0xfd7005060000
5029  munmap(0xfd7005057000, 36864)     = 0
5029  munmap(0xfd700522e000, 28224)     = 0
```

```
5029  mprotect(0xfd7005209000, 81920, PROT_NONE) = 0
5029  mmap(0xfd700521d000, 20480, PROT_READ|PROT_WRITE,
↪  MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x1ad000) = 0xfd700521d000
5029  mmap(0xfd7005222000, 48704, PROT_READ|PROT_WRITE,
↪  MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1, 0) = 0xfd7005222000
5029  mmap(0xfd7005060000, 1740800, PROT_READ|PROT_EXEC|0x10,
↪  MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0) = 0xfd7005060000
5029  close(3)                          = 0
5029  set_tid_address(0xfd700527c388)   = 5029
5029  set_robust_list(0xfd700527c040, 24) = 0
5029  rseq(0xfd700527c740, 0x21, 0, 0xd428bc00) = 0
5029  mprotect(0xfd700521d000, 12288, PROT_READ) = 0
5029  mprotect(0xc01ed355f000, 4096, PROT_READ) = 0
5029  mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS,
↪  -1, 0) = 0xfd7005279000
5029  mprotect(0xfd7005283000, 8192, PROT_READ) = 0
5029  prlimit64(0, RLIMIT_STACK, NULL, {rlim_cur=8192*1024,
↪  rlim_max=RLIM64_INFINITY}) = 0
5029  getrandom("\xa0\xa1\x92\xb7\x50\x5c\xd4\xd2", 8, GRND_NONBLOCK) = 8
5029  munmap(0xfd7005235000, 63707)     = 0
5029  getppid()                         = 5028
5029  rt_sigprocmask(SIG_BLOCK, [USR2], NULL, 8) = 0
5029  mmap(NULL, 4, PROT_READ|PROT_WRITE, MAP_SHARED, 4, 0) =
↪  0xfd7005278000
5029  close(4)                          = 0
5029  brk(NULL)                         = 0xc01ede8bb000
5029  brk(0xc01edea00000)               = 0xc01edea00000
5029  fstat(0, {st_mode=S_IFREG|0664, st_size=64, ...}) = 0
5029  read(0, "32 2 2 2 2 2 2\n100 5\n10\n1000 0.0"..., 4096) = 64
5029  kill(5028, SIGUSR1 <unfinished ...>
5028  <... rt_sigsuspend resumed>)      = ? ERESTARTNOHAND (To be
↪  restarted if no handler)
5029  <... kill resumed>)               = 0
5028  --- SIGUSR1 {si_signo=SIGUSR1, si_code=SI_USER, si_pid=5029,
↪  si_uid=1000} ---
5028  rt_sigreturn({mask=[USR1 CHLD]} <unfinished ...>
5029  rt_sigtimedwait([USR2] <unfinished ...>
5028  <... rt_sigreturn resumed>)       = -1 EINTR (Interrupted system
↪  call)
5028  write(1, "0.5\n", 4)              = 4
5028  kill(5029, SIGUSR2)               = 0
5029  <... rt_sigtimedwait resumed>, {si_signo=SIGUSR2, si_code=SI_USER,
↪  si_pid=5028, si_uid=1000}, NULL, 8) = 12 (SIGUSR2)
5029  kill(5028, SIGUSR1 <unfinished ...>
5028  rt_sigsuspend([], 8 <unfinished ...>
5029  <... kill resumed>)               = 0
5028  <... rt_sigsuspend resumed>)      = ? ERESTARTNOHAND (To be
↪  restarted if no handler)
5029  rt_sigtimedwait([USR2] <unfinished ...>
5028  --- SIGUSR1 {si_signo=SIGUSR1, si_code=SI_USER, si_pid=5029,
↪  si_uid=1000} ---
5028  rt_sigreturn({mask=[USR1 CHLD]})  = -1 EINTR (Interrupted system
↪  call)
5028  write(1, "20\n", 3)               = 3
5028  kill(5029, SIGUSR2)               = 0
5029  <... rt_sigtimedwait resumed>, {si_signo=SIGUSR2, si_code=SI_USER,
↪  si_pid=5028, si_uid=1000}, NULL, 8) = 12 (SIGUSR2)
5028  rt_sigsuspend([], 8 <unfinished ...>
5029  kill(5028, SIGUSR1 <unfinished ...>
5028  <... rt_sigsuspend resumed>)      = ? ERESTARTNOHAND (To be
↪  restarted if no handler)
```

```
5029  <... kill resumed>)               = 0
5028  --- SIGUSR1 {si_signo=SIGUSR1, si_code=SI_USER, si_pid=5029,
↪  si_uid=1000} ---
5028  rt_sigreturn({mask=[USR1 CHLD]} <unfinished ...>
5029  rt_sigtimedwait([USR2] <unfinished ...>
5028  <... rt_sigreturn resumed>)       = -1 EINTR (Interrupted system
↪  call)
5028  write(1, "10\n", 3)               = 3
5028  kill(5029, SIGUSR2)               = 0
5029  <... rt_sigtimedwait resumed>, {si_signo=SIGUSR2, si_code=SI_USER,
↪  si_pid=5028, si_uid=1000}, NULL, 8) = 12 (SIGUSR2)
5028  rt_sigsuspend([], 8 <unfinished ...>
5029  kill(5028, SIGUSR1)               = 0
5028  <... rt_sigsuspend resumed>)      = ? ERESTARTNOHAND (To be
↪  restarted if no handler)
5028  --- SIGUSR1 {si_signo=SIGUSR1, si_code=SI_USER, si_pid=5029,
↪  si_uid=1000} ---
5029  rt_sigtimedwait([USR2] <unfinished ...>
5028  rt_sigreturn({mask=[USR1 CHLD]})  = -1 EINTR (Interrupted system
↪  call)
5028  write(1, "100000\n", 7)           = 7
5028  kill(5029, SIGUSR2)               = 0
5028  rt_sigsuspend([], 8 <unfinished ...>
5029  <... rt_sigtimedwait resumed>, {si_signo=SIGUSR2, si_code=SI_USER,
↪  si_pid=5028, si_uid=1000}, NULL, 8) = 12 (SIGUSR2)
5029  kill(5028, SIGUSR1 <unfinished ...>
5028  <... rt_sigsuspend resumed>)      = ? ERESTARTNOHAND (To be
↪  restarted if no handler)
5029  <... kill resumed>)               = 0
5028  --- SIGUSR1 {si_signo=SIGUSR1, si_code=SI_USER, si_pid=5029,
↪  si_uid=1000} ---
5028  rt_sigreturn({mask=[USR1 CHLD]} <unfinished ...>
5029  rt_sigtimedwait([USR2] <unfinished ...>
5028  <... rt_sigreturn resumed>)       = -1 EINTR (Interrupted system
↪  call)
5028  write(1, "500\n", 4)              = 4
5028  kill(5029, SIGUSR2)               = 0
5029  <... rt_sigtimedwait resumed>, {si_signo=SIGUSR2, si_code=SI_USER,
↪  si_pid=5028, si_uid=1000}, NULL, 8) = 12 (SIGUSR2)
5028  rt_sigsuspend([], 8 <unfinished ...>
5029  kill(5028, SIGUSR1 <unfinished ...>
5028  <... rt_sigsuspend resumed>)      = ? ERESTARTNOHAND (To be
↪  restarted if no handler)
5029  <... kill resumed>)               = 0
5028  --- SIGUSR1 {si_signo=SIGUSR1, si_code=SI_USER, si_pid=5029,
↪  si_uid=1000} ---
5029  rt_sigtimedwait([USR2] <unfinished ...>
5028  rt_sigreturn({mask=[USR1 CHLD]})  = -1 EINTR (Interrupted system
↪  call)
5028  write(1, "0.05\n", 5)             = 5
5028  kill(5029, SIGUSR2)               = 0
5029  <... rt_sigtimedwait resumed>, {si_signo=SIGUSR2, si_code=SI_USER,
↪  si_pid=5028, si_uid=1000}, NULL, 8) = 12 (SIGUSR2)
5028  rt_sigsuspend([], 8 <unfinished ...>
5029  read(0, "", 4096)                 = 0
5029  kill(5028, SIGUSR1 <unfinished ...>
5028  <... rt_sigsuspend resumed>)      = ? ERESTARTNOHAND (To be
↪  restarted if no handler)
5029  <... kill resumed>)               = 0
5028  --- SIGUSR1 {si_signo=SIGUSR1, si_code=SI_USER, si_pid=5029,
↪  si_uid=1000} ---
```

```
5028  rt_sigreturn({mask=[USR1 CHLD]} <unfinished ...>
5029  rt_sigtimedwait([USR2] <unfinished ...>
5028  <... rt_sigreturn resumed>)       = -1 EINTR (Interrupted system
↪  call)
5028  write(1, "0\n", 2)                = 2
5028  kill(5029, SIGUSR2)               = 0
5029  <... rt_sigtimedwait resumed>, {si_signo=SIGUSR2, si_code=SI_USER,
↪  si_pid=5028, si_uid=1000}, NULL, 8) = 12 (SIGUSR2)
5029  munmap(0xfd7005278000, 4)         = 0
5028  rt_sigsuspend([], 8 <unfinished ...>
5029  exit_group(0)                     = ?
5029  +++ exited with 0 +++
5028  <... rt_sigsuspend resumed>)      = ? ERESTARTNOHAND (To be
↪  restarted if no handler)
5028  --- SIGCHLD {si_signo=SIGCHLD, si_code=CLD_EXITED, si_pid=5029,
↪  si_uid=1000, si_status=0, si_utime=0, si_stime=0} ---
5028  rt_sigreturn({mask=[USR1 CHLD]})  = -1 EINTR (Interrupted system
↪  call)
5028  wait4(5029, [{WIFEXITED(s) && WEXITSTATUS(s) == 0}], 0, NULL) =
↪  5029
5028  munmap(0xfde6c82a9000, 4)         = 0
5028  exit_group(0)                     = ?
5028  +++ exited with 0 +++
```